

Lecture # 4

Introduction to IBM PC Assembly Language

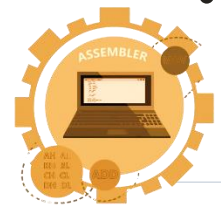


Engr. Fiaz Khan



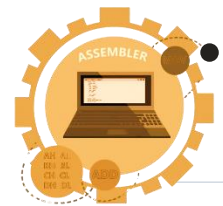
Assembly Language Syntax

- Assembly language programs are translated into machine language instructions by an assembler, such as the Microsoft Macro Assembler (MASM), which requires adherence to its specifications.
- Assembly language code is generally not case sensitive, but uppercase is typically used to differentiate code from text.
- Programs are composed of statements, each of which is either an instruction or an assembler directive.
- Instructions are translated into machine code, while directives instruct the assembler to perform specific tasks.
- Both instructions and directives consist of up to four fields: name, operation, operand(s), and comment.
- Fields must be separated by at least one blank or tab character and appear in the order mentioned above.



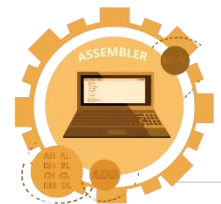
Assembly Language Syntax - Conti

- Example of an instruction:
- START: MOV CX,5 ;initialize counter
- Name: START, Operation: MOV, Operands: CX and 5, Comment: initialize counter
- Example of an assembler directive:
- MAIN PROC
- Name: MAIN, Operation: PROC
- This directive creates a procedure called MAIN.



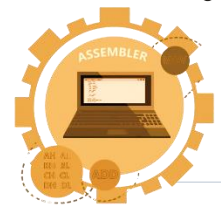
Name Field

- The name field in assembly language is utilized for instruction labels, procedure names, and variable names.
- Assemblers translate these names into memory addresses for referencing during program execution.
- Names can range from 1 to 31 characters in length and may consist of letters, digits, and certain special characters such as ?, . () _ \$ ' @.
- Embedded blanks within names are not permitted.
- If a period is included in the name, it must be the first character.
- Names cannot commence with a digit.
- The assembler treats names as case-insensitive, meaning it does not differentiate between uppercase and lowercase characters.



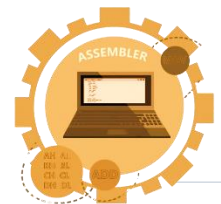
Operation Field

- In an instruction, the operation field holds a symbolic operation code, also known as an opcode.
- The assembler translates this symbolic opcode into a corresponding machine language opcode during the assembly process.
- Opcode symbols typically describe the function of the operation; examples include MOV, ADD, and SUB.
- In contrast, for an assembler directive, the operation field contains a pseudo-operation code, commonly referred to as a pseudo-op.
- Pseudo-ops do not get translated into machine code; instead, they instruct the assembler to perform specific tasks.
- An example of a pseudo-op is the PROC pseudo-op, which is used to define and create a procedure within the assembly code.



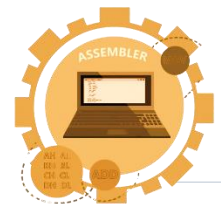
Operand Field

- In an instruction, the operand field specifies the data to be manipulated by the operation, and an instruction may have zero, one, or two operands.
- Example scenarios include:
- NOP: No operands; the instruction performs no action.
- INC AX: One operand; increments the contents of register AX by 1.
- ADD WORD1,2: Two operands; adds 2 to the contents of the memory word labeled WORD1.
- In a two-operand instruction, the first operand serves as the destination operand, indicating where the result is stored, although some instructions do not store the result. The second operand is the source operand, typically unaffected by the instruction.
- For an assembler directive, the operand field often provides additional information or parameters related to the directive itself.



Comment Field

- The comment field within a statement allows the programmer to provide explanations or descriptions of what the statement accomplishes.
- Comments in assembly language are initiated by a semicolon, and the assembler disregards any content following it on the same line.
- While comments are not mandatory, due to the low-level nature of assembly language, understanding a program becomes significantly challenging without them. Therefore, good programming practice emphasizes the inclusion of comments on nearly every line.
- Developing the skill of effective commenting is a process that improves with practice. It involves providing meaningful insights rather than stating the obvious.

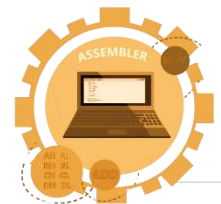


Comment Field - Conti

- Examples of good commenting practices include:
- Putting instructions into the context of the program.
- Avoiding redundant comments that merely restate the operation.
- Entire lines can serve as comments, which can be useful for creating visual separation or adding explanatory remarks.
- Example:
- ; Initialize registers

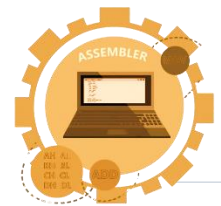
```
MOV AX, 0
```

```
MOV BX, 0
```



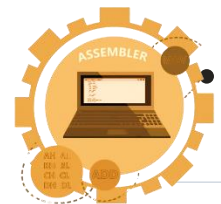
Program Data

- The processor operates exclusively on binary data, necessitating the assembler to convert all data representations into binary numbers.
- However, in assembly language programming, data can be expressed in various formats including binary, decimal, hexadecimal, and even characters.



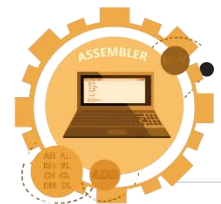
Numbers

- Binary numbers are written as a string of bits followed by the letter "B" or "b", for example, 1010B.
- Decimal numbers consist of a string of decimal digits, optionally ending with "D" or "d".
- Hexadecimal numbers must start with a decimal digit and end with "H" or "h", e.g., 0ABCH.
- Including a sign is optional for any of the mentioned number formats.
- Legal and illegal number examples for MASM include:
 - Decimal: 224, 124D (legal), illegal (contains a non-digit character)
 - Binary: 1010B (legal), illegal (doesn't end in "B")
 - Hexadecimal: 0ABCH (legal), illegal (doesn't begin with a decimal digit)



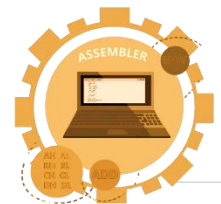
Characters

- Characters and character strings must be enclosed in single or double quotes, such as "A" or 'hello'.
- The assembler translates characters into their ASCII codes, making no difference between using character literals and their corresponding ASCII codes.
- Data-defining pseudo-ops are used in assembly language programming to define various data types:
 - DB: Define byte
 - DW: Define word
 - DD: Define doubleword (two consecutive words)
 - DQ: Define quadword (four consecutive words)
 - DT: Define tenbytes (ten consecutive bytes)



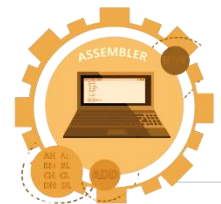
Variables

- Variables in assembly language serve the same purpose as they do in high-level languages, providing a means to store and manipulate data within a program.
- Each variable is assigned a data type and a memory address by the program.
- Data-defining pseudo-ops are utilized to define variables of different data types, and their meanings are outlined in Table 4.1.
- Each pseudo-op enables the allocation of one or more data items of the specified type within the program.
- In this section, we primarily employ the DB (define byte) and DW (define word) pseudo-ops to define byte variables, word variables, and arrays of bytes and words.
- Other data-defining pseudo-ops, such as DD, DQ, and DT, are utilized in Chapter 18 for operations involving multiple-precision and non-integer data types.



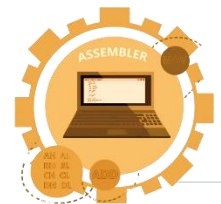
Byte Variables

- Assembler directives define byte variables using the **syntax: name DB initial value**.
- The pseudo-op DB stands for "Define Byte" in assembly language.
- For example, ALPHA DB 4 allocates a memory byte named ALPHA initialized to the value 4.
- Using a question mark (?) instead of an initial value reserves an uninitialized byte, e.g., BYT DB ?.
- The decimal range for initial values is -128 to 127 for signed interpretation and 0 to 255 for unsigned interpretation, fitting within a byte's range.



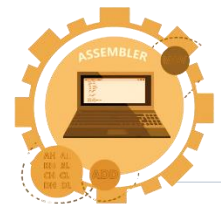
Word Variables

- The assembler directive for defining a word variable follows the **syntax: name dw initial value**.
- The pseudo-op DW represents "Define Word" in assembly language.
- For instance, WRD DW -2 allocates a memory word named WRD initialized to the value -2.
- Similar to byte variables, using a question mark (?) for the initial value denotes an uninitialized word.
- The decimal range for initial values is -32768 to 32767 for signed interpretation and 0 to 65535 for unsigned interpretation, within a word's range.



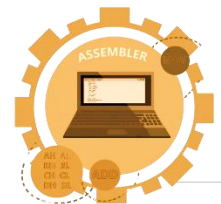
Arrays

- In assembly language, arrays are sequences of memory bytes or words.
- Arrays are defined using assembler directives such as DB for bytes and DW for words.
- Each element in the array is associated with a memory address.
- For example, `B_ARRAY DB 10H, 20H, 30H` defines a three-byte array with initial values 10h, 20h, and 30h.
- Memory addresses for array elements are sequential, starting from the base address.
- For instance, if `B_ARRAY` starts at address 0200h, the elements would be at addresses 200h, 201h, and 202h.
- Similarly, arrays of words are defined using DW.
- Each word in the array is associated with a memory address.
- For example, `W_ARRAY DW 1000, 40, 29887, 329` defines an array of four words with initial values 1000, 40, 29887, and 329.
- Memory addresses for word array elements are also sequential, with each word occupying two consecutive bytes.



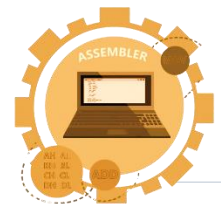
Arrays

- In assembly language, arrays are sequences of memory bytes or words.
- Arrays are defined using assembler directives such as DB for bytes and DW for words.
- Each element in the array is associated with a memory address.
- For example, `B_ARRAY DB 10H, 20H, 30H` defines a three-byte array with initial values 10h, 20h, and 30h.
- Memory addresses for array elements are sequential, starting from the base address.
- For instance, if `B_ARRAY` starts at address 0200h, the elements would be at addresses 200h, 201h, and 202h.
- Similarly, arrays of words are defined using DW.
- Each word in the array is associated with a memory address.
- For example, `W_ARRAY DW 1000, 40, 29887, 329` defines an array of four words with initial values 1000, 40, 29887, and 329.
- Memory addresses for word array elements are also sequential, with each word occupying two consecutive bytes.



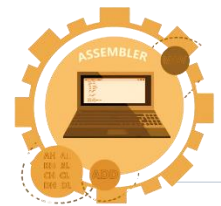
Low and High Bytes of Word

- Word Variable (WOHD1): This refers to a variable that holds a 16-bit value, meaning it's made up of two bytes.
- Low Byte: This is the first byte (or the byte at the lower memory address) of the word variable. In this case, the low byte of WOHD1 contains the value 34h.
- High Byte: This is the second byte (or the byte at the higher memory address) of the word variable. In this case, the high byte of WOHD1 contains the value 12h.
- Symbolic Addresses: These are labels that represent memory locations. The low byte has a symbolic address of WORD1, while the high byte's symbolic address is WORD1+1. This means that the high byte is located in the memory address right after the low byte.



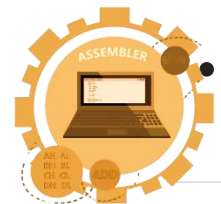
Character Strings

- Arrays of ASCII codes can represent strings of characters in assembly language.
- Initialization can be done using assembler directives like DB.
- For example, `LETTERS DB 'ABC'` initializes an array with ASCII values for the characters 'A', 'B', and 'C' (41h, 42h, and 43h).
- Both uppercase and lowercase characters are differentiated within strings.
- For instance, the string "abc" translates to ASCII values 61h, 62h, and 63h.
- Definitions can combine characters and numbers, allowing for versatile data representations.
- For instance, `MSG DB 'HELLO', 0AH, 0DH, '$'` initializes a message with ASCII values for "HELLO" followed by line feed, carriage return, and a dollar sign ('\$').



Named Constants

- In assembly language, using symbolic names for constants enhances code readability and understanding.
- The EQU (Equates) pseudo-op is used to assign a name to a constant quantity.
- Syntax: name EQU constant.
- For example, LF EQU 0AH assigns the name LF to the ASCII code for the line feed character.
- Subsequently, LF can be used in place of its ASCII value throughout the program.
- The assembler translates instructions using the symbolic name just as it would with the original constant value.
- The symbol on the right of an EQU statement can also be a string.
- Note: No memory is allocated for EQU names; they are purely symbolic.



MOV and XCHG

- The MOV instruction is fundamental in assembly language for transferring data between registers, between a register and a memory location, or for directly moving a number into a register or memory location.
- **Syntax: MOV destination, source.**
- Example: MOV AX, WORD1 means "Move the contents of WORD1 into AX." This replaces the content of register AX with the contents of the memory location WORD1, leaving the contents of WORD1 unchanged.
- Another example: MOV AX, BX transfers the content of register BX into AX, leaving BX unchanged.
- Additionally, MOV AH, 'A' directly moves the ASCII value of 'A' into the high byte of register AX, while the low byte remains unaffected.

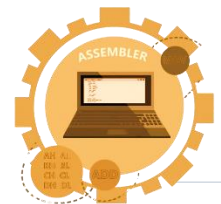


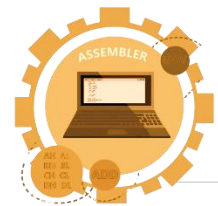
Table 4.2 Legal Combinations of Operands for MOV and XCHG

MOV

Source Operand	Destination Operand			
	General register	Segment register	Memory location	Constant
General register	yes	yes	yes	no
Segment register	yes	no	yes	no
Memory location	yes	yes	no	no
Constant	yes	no	yes	no

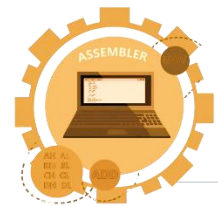
XCHG

Source Operand	Destination Operand	
	General register	Memory location
General register	yes	yes
Memory location	yes	no



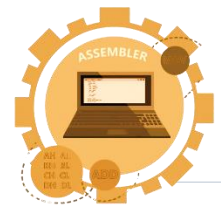
XCHG

Source Operand	Destination Operand	
	General register	Memory location
General register	yes	yes
Memory location	yes	no



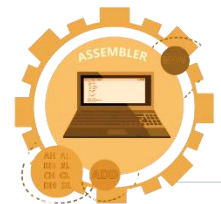
ADD, SUB, INC, and DEC - Conti

- The ADD and SUB instructions are used to perform addition or subtraction operations between registers, between a register and a memory location, or to add/subtract a number to/from a register or memory location.
- Syntax: ADD destination, source and SUB destination, source.
- Example: ADD WORD1, AX adds the contents of register AX to the memory word WORD1, storing the result in WORD1 while leaving AX unchanged.



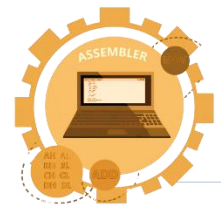
ADD, SUB, INC, and DEC - Conti

- Another example: SUB AX, DX subtracts the contents of register DX from AX, storing the difference in AX while leaving DX unchanged.
- Both INC (increment) and DEC (decrement) instructions add 1 or subtract 1, respectively, from a register or memory location.
- Syntax: INC destination and DEC destination.
- For instance, INC WORD1 adds 1 to the content of WORD1, while DEC BYTE1 subtracts 1 from the variable BYTE1.



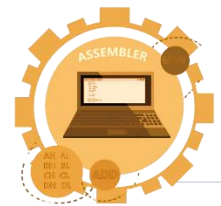
ADD, SUB, INC, and DEC - Conti

- NEG negates the contents of a destination by replacing it with its two's complement.
- Syntax: NEG destination.
- Example: NEG AX negates the content of AX by replacing it with its two's complement.
- Operand types must match in two-operand instructions, such as ADD, SUB, etc., ensuring compatibility between byte and word operations.



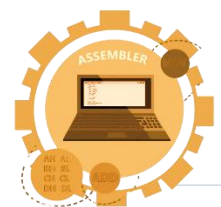
Program Structure

- Machine language programs comprise code, data, and stack segments, each occupying a distinct memory segment.
- Similarly, assembly language programs follow the same organization, with code, data, and stack structured as program segments.
- Program segments are translated into memory segments by the assembler.
- Simplified segment definitions introduced for Microsoft Macro Assembler (MASM) version 5.0 will be used, with further elaboration provided in Chapter 14, along with full segment definitions.



Memory Models

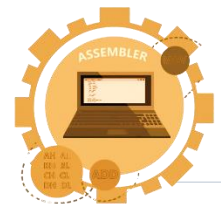
- The size of code and data a program can have is determined by specifying a memory model using the `.MODEL` directive.
- Syntax: `.MODEL memory_model`.
- Commonly used memory models include `SMALL`, `MEDIUM`, `COMPACT`, `LARGE`, and `HUGE`, each with specific characteristics detailed in Table 4.4.
- Unless dealing with a significant amount of code or data, the appropriate choice is often the `SMALL` model.
- The `.MODEL` directive should precede any segment definition in the program.
- Table 4.4 outlines the different memory models, specifying their descriptions and characteristics, such as code and data organization across segments, and limitations on array sizes.



Data Segment

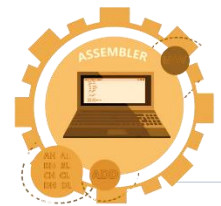
- The data segment of a program encompasses all variable definitions and often includes constant definitions.
- Constant definitions may be placed elsewhere in the program as no memory allocation is required.
- To declare a data segment, the `.DATA` directive is used, followed by variable and constant declarations.
- Syntax: `.DATA`, followed by variable and constant declarations.

```
.DATA  
WORD1 DW 2  
WORD2 DW 5  
MSG DB 'THIS IS A MESSAGE'  
MASK EQU 100100105
```



Stack Segment

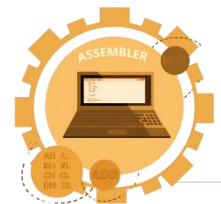
- The stack segment declaration allocates a portion of memory known as the stack area, reserved for storing the program's stack.
- The stack area should be sufficiently large to accommodate the stack at its maximum size.
- Syntax: `.STACK size`, where `size` is an optional parameter indicating the size of the stack area in bytes.
- For example, `.STACK 100H` allocates 100h (256) bytes for the stack area, which is typically sufficient for most applications.
- If the size parameter is omitted, the default behavior allocates 1 KB of memory for the stack area.



Code Segment

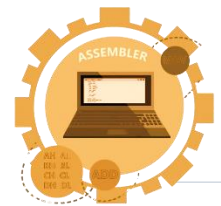
- The code segment contains a program's instructions, organized as procedures.
- Declaration syntax: `.CODE name`, where name is the optional name of the segment (not necessary for a SMALL program).
- Instructions within a code segment are structured as procedures.
- The basic procedure definition includes:
 - name PROC: Marks the beginning of a procedure.
 - name ENDP: Marks the end of a procedure.
- Here's an example of a code segment definition:

```
.CODE  
MAIN PROC  
; main procedure instructions  
MAIN ENDP  
; other procedures go here
```



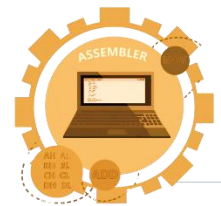
Putting It Together

```
.MODEL SMALL ; Specify the memory model as SMALL
.STACK 100h ; Allocate 100h bytes for the stack area
.DATA
; Data definitions go here
.CODE
MAIN PROC
; Instructions for the main procedure go here
MAIN ENDP
; Other procedures go here
END MAIN ; End of the program
```



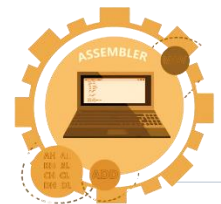
Input and Output Instructions

- CPUs communicate with peripherals via I/O registers called I/O ports, accessed using IN and OUT instructions for fast data transfer, common in game programs.
- However, most application programs avoid direct IN and OUT instructions due to varying port addresses among computer models and ease of programming I/O with manufacturer-provided service routines.
- Two categories of I/O service routines exist: BIOS routines (stored in ROM) and DOS routines. BIOS routines directly interact with I/O ports, while DOS routines, more complex, utilize BIOS routines for direct I/O operations.
- To invoke a DOS or BIOS routine, the INT (interrupt) instruction is used, with the format INT interrupt_number. For instance, INT 16h invokes a BIOS routine for keyboard input. Further details on the INT instruction are covered in Chapter 15.



The INT Instruction

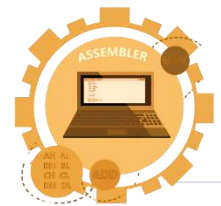
- The INT (interrupt) instruction is used to invoke DOS or BIOS routines.
- Its syntax is INT interrupt_number, where interrupt_number specifies a routine.
- For example, INT 16h invokes a BIOS routine for keyboard input.
- Chapter 15 provides detailed coverage of the INT instruction.
- INT 21h is commonly used to invoke various DOS functions, with specific functions requested by placing a function number in the AH register and then invoking INT 21h.
- The functions of interest are further elaborated.



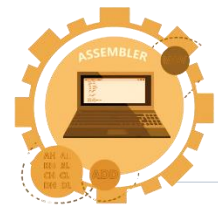
The INT Instruction

<i>Function number</i>	<i>Routine</i>
1	single-key input
2	single-character output✓
9	character string output

<i>ASCII code (Hex)</i>	<i>Symbol</i>	<i>Function</i>
7	BEL	beep (sounds a tone)
8	BS	backspace
9	HT	tab
A	LF	line feed (new line)
D	CR	carriage return (start of current line)



First Program



```
; Program Listing PGM4_1.ASM
TITLE PGM4_1: ECHO PROGRAM
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
    ; Display prompt
    MOV DL, '?'          ; Load '?' into DL (display prompt)
    INT 21H              ; Call DOS function to display character

    ; Input a character
    MOV AH, 1            ; Set AH to 1 (read character from standard input)
    INT 21H              ; Call DOS function to read character
    MOV BL, AL           ; Save character read into BL

    ; Go to a new line
    MOV AH, 2            ; Set AH to 2 (set cursor position)
    MOV DL, 0DH          ; Load carriage return character into DL
    INT 21H              ; Call DOS function to execute carriage return
    MOV DL, 0AH          ; Load line feed character into DL
    INT 21H              ; Call DOS function to execute line feed

    ; Display character
    MOV DL, BL           ; Load character from BL into DL
    INT 21H              ; Call DOS function to display character

    ; Return to DOS
    MOV AH, 4Ch          ; Set AH to 4Ch (DOS exit function)
    INT 21H              ; Call DOS function to exit to DOS
MAIN ENDP
END MAIN
```

Program Segment Prefix

- When a program is loaded into memory, DOS adds a 256-byte Program Segment Prefix (PSP) containing program information.
- DOS places the segment number of the PSP in both DS and ES registers before executing the program, causing DS not to contain the segment number of the data segment.
- To correct this, programs containing a data segment typically start with the instructions:

```
MOV AX, @DATA  
MOV DS, AX
```

- Here, @DATA represents the name of the data segment defined by the .DATA directive, and the assembler translates it into a segment number. Two instructions are necessary because a segment register cannot directly accept a segment number.

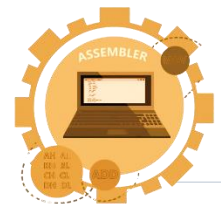


Program Segment Prefix

- With DS initialized, the "HELLO!" message can be printed by placing its address in DX and executing INT 21h:

```
LEA DX, MSG  
MOV AH, 9  
INT 21h
```

- The LEA instruction loads the address of the MSG string into DX, and INT 21h is used to display the string.





THANK YOU!

Do you have any questions?

ENGR. MUHAMMAD FIAZ

LECTURER COMPUTER SCIENCE

TELF/EF SET / ACEPT CERTIFIED

MICROSOFT ACADEMY TRAINER

ORACLE CERTIFIED PROFESSIONAL

GOOGLE CERTIFIED PROFESSIONAL

Lahore, Pakistan

+92-320-7617-093

muhammad.fiaz@cs.uol.edu.pk

<https://www.linkedin/in/fiazofficials>

